

Face Follow (PTZ)

1. Experiment Objective

PTZ pan-tilt + car body collaborative face following — Haar Cascade detects the face, the pan-tilt locks onto the face center with PD, the body turns to home based on the s1 angle, and the follow distance is maintained based on the face bbox width.

2. Experiment Principle

1. Key Terms

Term	Description
Visual Following	Pan-tilt tracking + coordinated body movement; the car actively turns and moves forward/back to follow
Error sharing (body_share)	The body takes on the pan-tilt angle $\times$ body_share homing task; the remainder is absorbed by the pan-tilt
Angular-rate feedforward (body_ff_gain)	When the body rotates, the pan-tilt compensates in the opposite direction synchronously, preventing the target from being dragged away by the body rotation
Distance Follow	PD control based on the face bbox width: small face (far) $\rightarrow$ move forward, large face (near) $\rightarrow$ move back
Lost-frame tolerance (lost_frame_threshold)	Only after N consecutive lost frames is the target judged lost, filtering occasional Haar detector misses
Haar Cascade	An OpenCV cascade classifier based on Haar features; uses a pre-trained XML model to detect faces

2. Technical Architecture

Node chain: state estimation (ekf_filter_node + camera_node) $\rightarrow$ face_follow_ptz_node $\rightarrow$ /cmd_ptz_angle (pan-tilt) + /cmd_vel (body)

Three-layer coordination: PTZ PD (30 Hz) + body angular-velocity PD (10 Hz) + distance-follow PD (10 Hz)

3. Core Topics Quick Reference

Firmware Uplink

Topic Name	Message Type	QoS	Source
/camera/image_raw	Image	best_effort	ESP32 camera
/odom	Odometry	best_effort	Odometry (needed by the body control)

Firmware Downlink

Topic Name	Message Type	QoS	Description
/cmd_ptz_angle	Vector3	best_effort + keep_last(1)	x: [-90,90]° horizontal, y: [-90,20]° tilt
/cmd_vel	Twist	best_effort + keep_last(1)	angular.z: turning, linear.x: forward/back

ROS2 Internal

Topic Name	Message Type	QoS	Source	Description
/joint_states	JointState	reliable	State estimation layer	Current pan-tilt angle feedback

3. Experiment Steps

1. Hardware Preparation

Hardware	Requirement
PTZ version	☒
No-PTZ version	☐
Required peripherals	ESP32 camera (pan-tilt) + PTZ servo

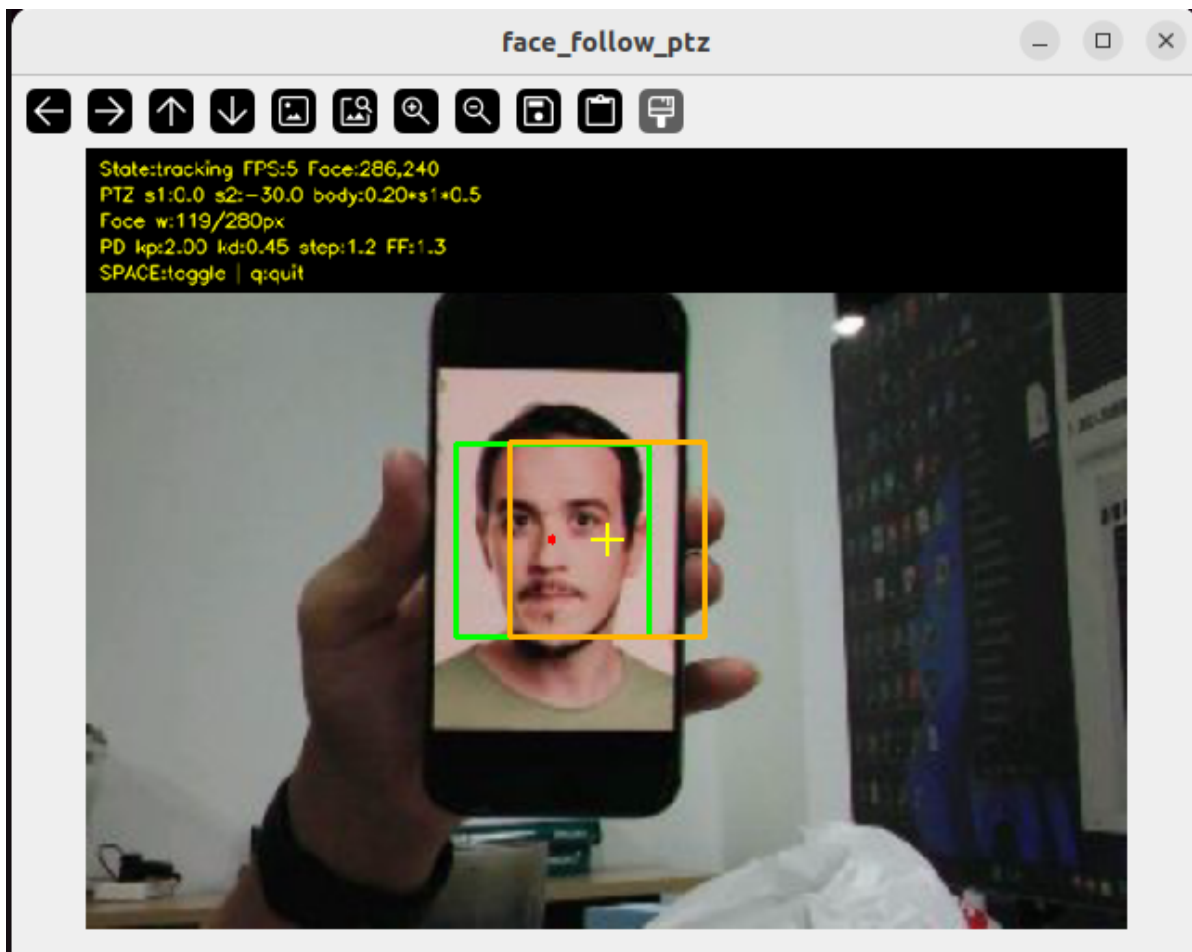
2. Start Agent + Camera + Joint Model (Terminal 1)

```
ros2 launch microros_v2_bringup car_base.launch.py
```

```
yahboom@yahboom:~$ ros2 launch microros_v2_bringup car_base.launch.py
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-07-28-15-04-46-269642-yahboom-579370
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [micro_ros_agent-1]: process started with pid [579411]
[INFO] [camera_driver-2]: process started with pid [579413]
[INFO] [robot_state_publisher-3]: process started with pid [579415]
[INFO] [joint_state_publisher-4]: process started with pid [579417]
[INFO] [ekf_node-5]: process started with pid [579419]
[micro_ros_agent-1] [1785222287.404236] info | UDPv4AgentLinux.cpp | init | running... | port: 8090
[robot_state_publisher-3] [INFO] [1785222288.847816851] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-3] [INFO] [1785222288.848013042] [robot_state_publisher]: got segment base_link
[robot_state_publisher-3] [INFO] [1785222288.848028942] [robot_state_publisher]: got segment bwheel_Link
[robot_state_publisher-3] [INFO] [1785222288.848037122] [robot_state_publisher]: got segment cam1_Link
[robot_state_publisher-3] [INFO] [1785222288.848043874] [robot_state_publisher]: got segment cam2_Link
[robot_state_publisher-3] [INFO] [1785222288.848050538] [robot_state_publisher]: got segment cam3_Link
[robot_state_publisher-3] [INFO] [1785222288.848057264] [robot_state_publisher]: got segment imu_frame
[robot_state_publisher-3] [INFO] [1785222288.848063747] [robot_state_publisher]: got segment laser_frame
[robot_state_publisher-3] [INFO] [1785222288.848070240] [robot_state_publisher]: got segment lfwheel_Link
[robot_state_publisher-3] [INFO] [1785222288.848076888] [robot_state_publisher]: got segment rfwheel_Link
[joint_state_publisher-4] [INFO] [1785222289.372349515] [joint_state_publisher]: Waiting for robot_description to be published on the robot_description
[camera_driver-2] [INFO] [1785222289.71611193] [esp32_ip_camera_publisher]: camera node started, camera_url: http://192.168.12.37:81/stream
[camera_driver-2] [WARN] [1785222292.785741414] [esp32_ip_camera_publisher]: Camera not opened, trying reconnect... (next retry in 1.0s)
[camera_driver-2] [INFO] [1785222294.072701944] [esp32_ip_camera_publisher]: IP camera stream opened successfully
```

3. Start Face Follow (Terminal 2)

```
ros2 launch microros_v2_ptz_visual_control_pkg face_follow_ptz.launch.py
```



4. Operation Instructions

Workflow:

The architecture is the same as Color Follow: Haar detection is done in the image callback, and the PTZ control (30 Hz) and body control (10 Hz) dual timers run independently.

1. **Face detection (Haar Cascade):** after decoding, the image is scaled to 640×480, converted to grayscale, and the OpenCV Haar cascade classifier `detectMultiScale` detects faces (`face_min_width / face_min_height = 80` px). The largest face is taken as the primary target, yielding the face center (cx, cy) and bbox width.
2. **N-frame lost-frame tolerance:** the target is marked lost and the body frozen only after consecutive no-face detections reach `lost_frame_threshold` (5 frames).
3. **Three-layer coordinated control:**
 - **PTZ PD (30 Hz):** normalized error → PD (`pd_kp=2.0`, `pd_kd=0.45`, higher gains for face movement) → angular-rate feedforward compensation → publish `/cmd_ptz_angle`
 - **Body angular-velocity PD (10 Hz):** `s1 × body_share` (0.5) → PD (`angular_kp=0.2`, `angular_kd=0.06`) → `/cmd_vel.angular.z`
 - **Distance follow (10 Hz):** deviation between the face bbox width and `target_face_width` (280 px) → PD (`distance_kp=0.008`, `distance_kd=0.003`) → `/cmd_vel.linear.x`

Interaction:

Key	Function	Description
Space	idle ↔ tracking	Toggles the follow state; stops in idle
i / I	Back to idle	Stops PTZ and body control
q / Q / ESC	Exit	Stops, then exits with <code>os._exit(0)</code>

Safety and Edge Cases:

- If the classifier fails to load, an error is logged when the Haar XML file is unavailable; the node does not crash
- Target lost ≥ 5 frames: marked lost, PD reset, body frozen (zero velocity)
- s1 is zeroed within 3° , avoiding fine-tuning jitter
- Pan-tilt angle limiting: horizontal $[-90^\circ, 90^\circ]$, tilt $[-90^\circ, 20^\circ]$
- Velocity limiting: angular velocity ± 0.15 rad/s, linear velocity ± 0.12 m/s
- On exit, stop first, then `os._exit(0)`

5. How to Stop

Press `Ctrl+C` to stop Terminal 2 → Terminal 1 in order.

4. Experiment Observations

- **idle state:** The frame shows face detection in real time, the pan-tilt does not move, and the body is stationary
- **tracking + face detected:** The pan-tilt locks onto the face center with PD, the body turns automatically with s1, and simultaneously moves forward/back according to the face size to maintain a bbox width of about 280 px
- **Person approaches (face grows):** the car moves back to maintain the distance
- **Person moves away (face shrinks):** the car moves forward to approach
- **Face deviates from center:** the body takes on 50% of the turning, the pan-tilt takes the rest + feedforward compensates for the body rotation
- **Profile face / brief occlusion:** occasional Haar detection misses → filtered by the N-frame tolerance
- **Face gone > 5 frames:** the body freezes, waiting for it to reappear

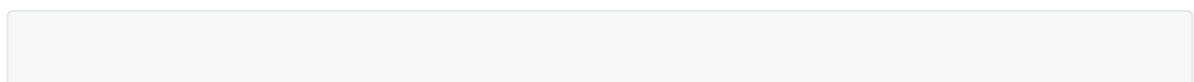
5. Program Analysis

Source code paths:

- Launch:
`~/microros_ws/src/microros_v2_ptz_visual_control_pkg/launch/face_follow_ptz.launch.py`
- Node:
`~/microros_ws/src/microros_v2_ptz_visual_control_pkg/microros_v2_ptz_visual_control_pkg/face_follow_ptz_node.py`

1. Core Node Analysis

`FaceFollowPtzNode` completes the parameter declaration, face classifier loading, subscriber/publisher creation, and dual-timer initialization in `__init__`:



```
# Source file:
~/microros_ws/src/microros_v2_ptz_visual_control_pkg/microros_v2_ptz_visual_control_pkg/face_follow_ptz_node.py
```

```
class FaceFollowPtzNode(Node):

    def __init__(self) -> None:
        super().__init__(DEFAULT_NODE_NAME)
        self._tgt_lock = threading.Lock()
        self._cv_bridge = CvBridge()
        self._latest_frame: Optional[np.ndarray] = None

        self._state = STATE_IDLE
        self._shutdown = False

        # Target state
        self._has_target = False
        self._lost_frames = 0; self._lost_frame_threshold = 5
        self._target_x: Optional[float] = None
        self._target_y: Optional[float] = None
        self._target_r: Optional[float] = None # width of the detected face box

        # PTZ
        self._h_deg = 0.0; self._p_deg = 0.0
        self._prev_err_x = 0.0; self._prev_err_y = 0.0
        self._prev_ctrl_t = 0.0; self._last_sx = 0.0; self._last_sy = 0.0

        # Face detection
        self._face_cascade = None

        # Body PD state
        self._prev_s1 = 0.0; self._prev_body_t = 0.0; self._body_share = 0.5
        self._prev_s1_eff = 0.0
        self._last_body_az = 0.0; self._body_ff_gain = 1.3

        # Distance-follow state
        self._target_face_w = 80.0
        self._prev_r_err = 0.0; self._prev_linear_t = 0.0

        self._fps = 0.0; self._fps_n = 0; self._fps_t = time.perf_counter()
        self._win_ok = False; self._last_print_t = 0.0

        self._declare_params()
        self._load_params()
        self._h_deg = self._init_h_deg; self._p_deg = self._init_p_deg

        # Load the face cascade classifier
        self._reload_face_cascade()

        video_qos = QoSProfile(
            history=QoSHistoryPolicy.KEEP_LAST,
            depth=1,
            reliability=QoSReliabilityPolicy.BEST_EFFORT,
            durability=QoSDurabilityPolicy.VOLATILE
        )

        self._sub = self.create_subscription(
            Image, self._img_topic, self._on_image, video_qos,
```

```

)
self._ptz_pub = self.create_publisher(Vector3, self._ptz_topic, 10)
self._cmd_pub = self.create_publisher(Twist, self._cmd_topic, 10)

# Timers
self._ptz_timer = self.create_timer(
    1.0 / max(self._ptz_rate, MINIMUM_TIMER_RATE_HZ), self._ptz_tick,
)
self._body_timer = None; self._rebuild_body_timer(False)
self._key_timer = None
if self._show_win:
    self._init_debug_win()
    self._key_timer = self.create_timer(0.05, self._poll_keys)

self._init_ptz_timer = None; self._init_ptz_remain = 0
if self._pub_init: self._start_init_ptz()

self._param_cb =
self.add_on_set_parameters_callback(self._on_param_update)
self.get_logger().info(f"Face follow PTZ+body started. SPACE=toggle,
q=quit. "
                        f"ptz={self._ptz_rate:.0f}Hz body=
{self._body_rate:.0f}Hz")

```

Face detection method `_detect_face`: converts the frame to grayscale, detects faces with `CascadeClassifier.detectMultiScale`, takes the largest face as the primary target, and computes the center coordinates and bbox width to update the target state.

PTZ control method `_ptz_tick`: the same PD control structure as Color Follow — normalized error, PD step computation, angular-rate feedforward compensation, and dual EMA smoothing before publishing `/cmd_ptz_angle`. Face following uses higher gains (`pd_kp=2.0`, `pd_kd=0.45`) to accommodate fast face movement.

Body control method `_body_tick`: `s1` is scaled by `body_share` (0.5), then PD computes the angular velocity; the deviation between the face bbox width and `target_face_width` is fed into PD for the linear velocity, and the merged `/cmd_vel` is published.

2. Key Parameters

Parameter definition file:

```
~/microros_ws/src/microros_v2_ptz_visual_control_pkg/launch/face_follow_ptz.launch.py
```

Parameter	Type	Default Value	Description
<code>pd_kp</code> / <code>pd_kd</code>	float	2.0 / 0.45	Pan-tilt PD gains (higher than Color Follow; faces move fast)
<code>ptz_control_rate</code>	float	30.0 Hz	Pan-tilt control rate
<code>body_control_rate</code>	float	10.0 Hz	Body control rate
<code>deadband_x</code> / <code>deadband_y</code>	float	60.0 / 60.0 px	Deadband (larger, tolerant of face-box jitter)
<code>angular_kp</code> / <code>angular_kd</code>	float	0.2 / 0.06	Body angle PD gains
<code>body_share</code>	float	0.5	Ratio of the error taken on by the body
<code>body_ff_gain</code>	float	1.3	Angular-rate feedforward gain
<code>target_face_width</code>	float	280.0 px	Distance-follow target face width
<code>distance_kp</code> / <code>distance_kd</code>	float	0.008 / 0.003	Distance-follow PD gains
<code>distance_tolerance_fw</code>	float	35.0 px	Distance-follow deadband
<code>max_angular_z</code>	float	0.15 rad/s	Maximum turning angular velocity
<code>max_linear_x</code>	float	0.12 m/s	Maximum linear velocity
<code>lost_frame_threshold</code>	int	5	Lost-frame tolerance threshold

6. Frequently Asked Questions

Problem	Cause	Solution
The distance oscillates far and near	The bbox width fluctuates because of Haar box jitter	Keep facing the camera; increase <code>distance_tolerance_fw</code> (e.g., 50)
Profile face loses tracking	Haar Cascade is sensitive to frontal faces	Keep the frontal face facing the camera
The body turns too aggressively	<code>angular_kp</code> too large (0.2)	Reduce <code>angular_kp</code> (e.g., 0.1)